

Installation on EC2

12 August 2025 19:40

Launch a new EC2

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

Terraform Server

Add additional tags

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

RecentsQuick Start

RecentsQuick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Del

del

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

ami-0662f4965dfc70aca (64-bit (x86)) / ami-09ed9bca6a01cd74a (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.medium

Family: t3 2 vCPU 4 GiB Memory Current generation: true

On-Demand Windows base pricing: 0.0704 USD per Hour

On-Demand SUSE base pricing: 0.1083 USD per Hour

On-Demand RHEL base pricing: 0.0808 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0555 USD per Hour

On-Demand Linux base pricing: 0.052 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

Create key pair

Key pair name

Key pairs allow you to connect to your instance securely.

terraformkey

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA

RSA encrypted private and public key pair

☐ ED25519

ED25519 encrypted private and public key pair

Private key file format

☒ .pem

For use with OpenSSH

☐ .ppk

For use with PuTTY

Cancel

Create key pair

2. SSH to server using terminal.

3. Got to google, click on Install Terraform

Google

install terraform

AI ModeAllVideosImagesShort videosForumsShoppingMore

Tools

4.

HashiCorp Developer

https://developer.hashicorp.com/.../Tutorials/AWS

Install Terraform

New Section 1 Page 1

Google

install terraform

AI Mode All Videos Images Short videos Forums Shopping More Tools

4. HashiCorp Developer
https://developer.hashicorp.com/.../Tutorials/AWS
Install Terraform
Install Terraform on Mac, Linux, or Windows by downloading the binary or using a package manager (Homebrew or Chocolatey). Then create a Docker container ...
AI Overview

developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli

AWS Diagram Draw Inbox - Zoho Mail Mail - Koustubh Juv... FortuneCloud Teams Learnings Platforms IBM Full Stack Softw...

HashiCorp Terraform Install Tutorials Documentation Registry Try Cloud

< AWS
6 tutorials
Infrastructure as Code
Install
Create
Manage
Destroy
HCP Terraform
Resources
Tutorial Library
Certifications
Community Forum

Install Terraform

HashiCorp distributes Terraform as an executable CLI that you can install on supported operating systems, including Microsoft Windows, macOS, and several Linux distributions. You can also compile the Terraform CLI from source if a pre-compiled binary is not available for your system.

1 Package manager Manual installation

If you use a package manager to install software on your macOS, Windows, or Linux system, you can use it to install Terraform.

Homebrew on macOS Chocolatey on Windows Linux 2

HashiCorp maintains and signs packages for the following Linux distributions. Use the built-in package management system for your Linux distribution to install Terraform.

Ubuntu/Debian

Follow all installation steps given

Ensure that your system is up to date and that you have installed the `gnupg` and `software-properties-common` packages. You will use these packages to verify HashiCorp's GPG signature and install HashiCorp's Debian package repository.

```
$ sudo apt-get update && sudo apt-get install -y gnupg software-properties-common
```

Install HashiCorp's GPG key.

```
$ wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
```

Verify the GPG key's fingerprint.

```
$ gpg --no-default-keyring \
--keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg \
--fingerprint
```

The `gpg` command reports the key fingerprint:

```
/usr/share/keyrings/hashicorp-archive-keyring.gpg
-----
pub  rsa4096 XXXX-XX-XX [SC]
AAAA AAAA AAAA AAAA
uid   [ unknown] HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>
sub  rsa4096 XXXX-XX-XX [E]
```

Add the official HashiCorp repository to your system.

```
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*/etc/os-release ||
lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Update apt to download the package information from the HashiCorp repository.

```
$ sudo apt update
```

Install Terraform from the new repository.

```
$ sudo apt-get install terraform
```

Terraform versions and compatibility

HashiCorp regularly releases new versions of Terraform with new features and bug fixes. HashiCorp maintains compatibility between Terraform versions, so a Terraform configuration written for one version of Terraform should continue to work with any later minor version update. Refer to the [Terraform compatibility promise](#) for more details.

Verify the Installation

Verify that the installation worked by opening a new terminal session and listing Terraform's available subcommands.

```
$ terraform -help
```

Usage: terraform [global options] <subcommand> [args]
The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.
Main commands:
##...

Add -help to any Terraform command to learn more about what it does and available options.

```
$ terraform plan -help
```

Enable tab completion

If you use either Bash or Zsh as your command line shell, you can enable tab completion for Terraform commands. To enable autocomplete, first ensure that a configuration file exists for your chosen shell.

```
BashZsh
$ touch ~/.bashrc
```

Then install the autocomplete package.

```
$ terraform -install-autocomplete
```

After installing autocomplete support, you will need to restart your shell to enable it.

INSHORT HISTORY:

```
16 sudo apt-get update && sudo apt-get install -y gnupg software-properties-common
17 wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
18 gpg --no-default-keyring --keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg --fingerprint
19 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release
| | Isb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
20 sudo apt update
21 sudo apt-get install terraform
22 terraform -help
23 terraform plan -help
24 touch ~/.bashrc
25 terraform -install-autocomplete
```

The screenshot shows the Terraform Registry page for the AWS provider. The browser address bar shows the URL: registry.terraform.io/providers/hashicorp/aws/latest. The page has a blue header with the Terraform logo, 'Registry', a search bar, and links for 'Browse', 'Publish', and 'Sign-in'. Below the header, the breadcrumb navigation shows 'Providers / hashicorp / aws / Version 6.8.0'. The main content area has tabs for 'Overview' and 'Documentation', with 'Documentation' being the active tab. A 'USE PROVIDER' button is also visible. The AWS provider logo and name are prominently displayed, along with the text 'Official by HashiCorp' and 'Public Cloud'. A description states: 'Lifecycle management of AWS resources, including EC2, Lambda, EKS, ECS, VPC, S3, RDS, DynamoDB, and more. This provider is maintained internally by the HashiCorp AWS Provider team.' Below this, it shows 'VERSION 6.8.0', 'PUBLISHED 5 days ago', and a link to 'SOURCE CODE hashicorp/terraform-provider-aws'. On the right side, a 'Provider Downloads' table is shown, with a red circle highlighting the 'All versions' dropdown menu. The table lists downloads for 'this week' (36.2M), 'this month' (51.8M), 'this year' (1.0B), and 'over all time' (4.7B).

Downloads this week	36.2M
Downloads this month	51.8M
Downloads this year	1.0B
Downloads over all time	4.7B

Click on Documentation

The screenshot shows the AWS Documentation page for the Terraform provider. The left sidebar has a search bar and a list of navigation links: 'aws provider', 'Guides', 'Functions', 'ACM (Certificate Manager)', 'ACM PCA (Certificate Manager Private Certificate Authority)', 'AMP (Managed Prometheus)', 'API Gateway', 'API Gateway V2', 'Account Management', 'Amazon Q Business', and 'Amplify'. The main content area is titled 'Example Usage' and shows the Terraform configuration for the AWS provider. The text 'Terraform 0.13 and later:' is displayed above the code. The code is as follows:

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}

# Create a VPC
resource "aws_vpc" "example" {
  cidr_block = "10.0.0.0/16"
}
```

 A 'Copy' button is located to the right of the code block.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}

# Create a VPC
resource "aws_vpc" "example" {
  cidr_block = "10.0.0.0/16"
}
```

The screenshot shows the AWS documentation search interface. The search bar contains 'aws-instance'. Below the search bar, it says '19 matching results'. The search results are categorized under 'EC2 (Elastic Compute Cloud)', 'Resources', and 'Data Sources'. Under 'Resources', the item 'aws_instance' is highlighted. To the right of the search results, there is a snippet of Terraform code for an AWS instance named 'web'. The code defines the 'ami' as 'resolve:ssm:/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2' and the 'instance_type' as 't3.micro'. The 'tags' list includes 'Name = "HelloWorld"'. A 'Copy' button is visible next to the code snippet.

EC2

Instances

i-09da10b0180bfd685

Details

Status and alarms

Monitoring

▼ Instance details

Info

AMI ID

ami-0662f4965dfc70aca

Instance summary for i-09da10b0180bfd685 (Terraform Server)

Info

Updated about 3 hours ago

Instance ID

i-09da10b0180bfd685

Public IPv4 address

15.165.204.205 | [open address](#)

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-44-85.ap-northeast-2.compute.internal

Instance type

t3.medium

IPV6 address

-

Hostname type

IP name: ip-172-31-44-85.ap-northeast-2.compute.internal

Answer private resource DNS name

IPv4 (A)

[Alt+S]

Asia Pacific

i-09da10b0180bfd685

Instance summary for i-09da10b0180bfd685 (Terraform Server)

Updated 1 minute ago

Instance ID

i-09da10b0180bfd685

IPV6 address

-

Hostname type

IP name: ip-172-31-44-85.ap-northeast-2.compute.internal

Answer private resource DNS name

IPv4 (A)

Auto-assigned IP address

15.165.204.205 [Public IP]

Public IPv4 address

15.165.204.205 | [open address](#)

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-44-85.ap-northeast-2.compute.internal

Instance type

t3.medium

VPC ID

vpc-0e03e7944773115da

United States

N. Virginia

us-east-1

Ohio

us-east-2

N. California

us-west-1

Oregon

us-west-2

Asia Pacific

Mumbai

ap-south-1

Osaka

ap-northeast-3

Seoul

ap-northeast-2

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Canada

Central

ca-central-1

Europe

Frankfurt

eu-central-1

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

eu-west-2

Paris

eu-west-1

Stockholm

eu-north-1

Amsterdam

eu-west-3

Manchester

eu-west-1

Madrid

eu-west-1

Rome

eu-west-1

Sa Paulo

sa-east-1

Singapore

ap-southeast-1

Sydney

ap-southeast-2

Tokyo

ap-northeast-1

Yokohama

ap-northeast-2

London

New Section 1 Page 4

```

GNU nano 7.2 main.tf *
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "ap-northeast-2"
}

#Launch Ec2
resource "aws_instance" "myec2" {
  ami           = "ami-0662f4965dfc70aca"
  instance_type = "t3.medium"

  tags = {
    Name = "MyEC2byTerraform"
  }
}

```

Save it.

Then

```

ubuntu@TerraformServer:~$ mkdir myterraform
ubuntu@TerraformServer:~$ cd myterraform/
ubuntu@TerraformServer:~/myterraform$ ls
ubuntu@TerraformServer:~/myterraform$ nano main.tf

ubuntu@TerraformServer:~/myterraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.8.0...
- Installed hashicorp/aws v6.8.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@TerraformServer:~/myterraform$ ls
main.tf
ubuntu@TerraformServer:~/myterraform$ ls -la
. . . .terraform .terraform.lock.hcl main.tf

```

```

ubuntu@TerraformServer:~/myterraform$ terraform plan

Planning failed. Terraform encountered an error while generating this plan.

Error: No valid credential sources found

    with provider["registry.terraform.io/hashicorp/aws"],
    on main.tf line 11, in provider "aws":
    11: provider "aws" {

Please see https://registry.terraform.io/providers/hashicorp/aws
for more information about providing credentials.

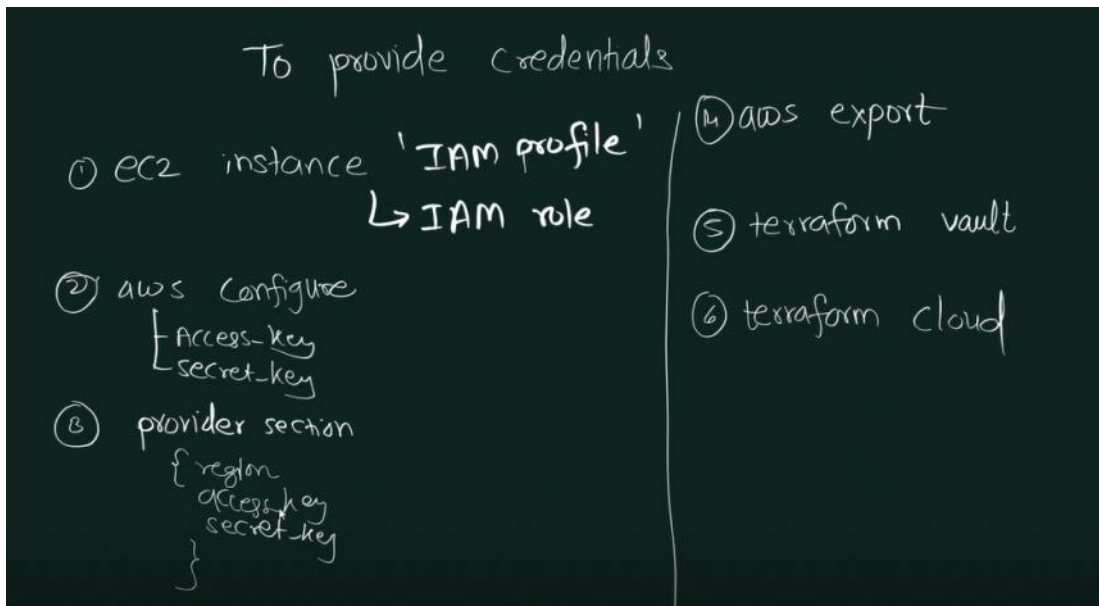
Error: failed to refresh cached credentials, no EC2 IMDS role found, operation error ec2imds: GetMetadata, http response error StatusCode: 404
request to EC2 IMDS failed

ubuntu@TerraformServer:~/myterraform$

```

Now this error you will see? Why

Because without credential AWS will not accept request to perform operations. So we need to give credentials to terraform. There are different ways to give credentials, let's use one of them "through file."



Through file means we are using 3rd one,

SO let's create first access key.

Go to aws page.

On profile --> go to Security Credentials

Create access key

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

Secret access key

AKIAJGGILTIU4N5K5J47

***** [Show](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#)

[Done](#)

Download it, Save it! For now you can copy from here and paste in main.tf file, lets do this...

```
GNU nano 7.2 main.tf *
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "ap-northeast-2"
  access_key = "AKIAJGGILTIU4N5K5J47"
  secret_key = "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"
}

#Launch Ec2
resource "aws_instance" "myec2" {
  ami = "ami-0662f4965dfc70aca"
  instance_type = "t3.medium"

  tags = {
    Name = "MyEC2byTerraform"
  }
}
```

Save it, and again...

```
ubuntu@TerraformServer:~/myterraform$ terraform plan

Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_instance.myec2 will be created
+ resource "aws_instance" "myec2" {
  + ami = "ami-0662f4965dfc70aca"
  + arn = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + disable_api_stop = (known after apply)
  + disable_api_termination = (known after apply)
}
```

```
ubuntu@TerraformServer:~/myterraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource
+ create

Terraform will perform the following actions:

# aws_instance.myec2 will be created
+ resource "aws_instance" "myec2" {
  + ami = "ami-0662f4965dfc70aca"
  + arn = (known after apply)
}
```

```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.myec2: Creating...
aws_instance.myec2: Still creating... [00m10s elapsed]
aws_instance.myec2: Creation complete after 12s [id=i-055b06a55d83916f1]
```

Instances (1/2) [Info](#)

Last updated
less than a minute ago



[Connect](#)

[Instance state](#)

[Actions](#)

[Launch instances](#)

[All states](#)

☐	Name	Instance ID	Instance state	Instance type
☐	Terraform Server	i-09da10b0180bfd685	Running	t3.medium
☒	MyEC2byTerraform	i-055b06a55d83916f1	Running	t3.medium

From AMI of Terraform Server we successfully created **MyEC2byTerraform** named server, which automatically created! For now it will automatically take default **SecurityGroup**.
If you try to connect this instance, it will show error
So got to it's Security group and add SSH 22 Port, then connect, it will connect.